This approach leverages event delegation, allowing clicks on any `<li>`—including those added dynamically—to be handled by the parent's listener. This works because most events in the DOM, such as `click`, bubble up from the target element to their ancestors, enabling the parent to intercept and handle these events.

However, not all events in JavaScript can be delegated because some do not bubble up through the DOM tree. Events such as `focus`, `blur`, `mouseenter`, `mouseleave`, `scroll`, and `resize` do not bubble and therefore cannot be captured by a parent's event listener using delegation. For these events, listeners must be attached directly to the target elements.

In large or complex DOMs, the parent handler must identify the correct target for every event, which can introduce performance overhead, especially with many child elements or deep nesting. Delegated handlers may also become complex when dealing with many different child types or behaviors, making code harder to maintain. Since event delegation relies on event bubbling, issues may arise if other listeners higher in the DOM also handle the event or if `event.stopPropagation()` is used, which can interfere with expected behavior and complicate debugging.

Dynamic DOM changes during event propagation, such as removing a target or ancestor, can cause unexpected behavior or errors, particularly in complex applications. Additionally, some features available with direct binding—like one-time (once) or passive event listeners—may be harder or less efficient to implement with delegation. If the parent container has no relevant child elements, the listener may still be triggered by clicks on empty areas, requiring extra checks to avoid unnecessary processing.

For this topic, are you able to answer these follow-up/probing questions?

- "How does event delegation leverage event bubbling in the DOM?"
- "What are the main benefits of using event delegation instead of attaching individual listeners to each child element?"
- "Can you describe a scenario where event delegation is particularly advantageous?"
- "Are there any events that cannot be delegated? If so, why?"